

Reflection in C++26 (P2996)

Frantz Maerten, Look Up Geoscience

June 11, 2026

What is reflection?

"The ability of software to expose its internal structure."

Static reflection: the **compiler** exposes structure at compile time

- Inspection of types, members, functions, and annotations
- Automatic C++ code generation at compile time
- Expressive compile-time metaprogramming without macros

P2996 — one of the **largest** proposals in C++ history since the introduction of templates.

Difference with Introspection

- Introspection = **reading** (observe only)
- Reflection = **reading** + **acting** (observe and manipulate)

→ **Reflection** is the broader capability: **introspection** + the ability **to act** on what you discovered.

As we will see, C++ P2996 is reflection 💪 not introspection.

Why should you care?

One C++ description → *everything*:

- **Automatic serialization / deserialization**
- **Generic struct printing / debugging**
- **Compile-time generation of types via splicers**
- **Doc generation** (md, html...)
- Annotations (P3394, paired with reflection) let you attach compile-time metadata to declarations and read it back via reflection, supporting things like validation rules or serialization hints.
- ...

The two operators in one picture

```
constexpr std::meta::info info = ^^Circle;           // reflect-on a type
typename [: info :] c = {.name = "c", .radius = 1.0}; // splice it back into code
// also works on members, expressions, namespaces, templates...
```

`^^` aka - cat-ears operator

- Everything that crosses the boundary is `constexpr` / `consteval`
- `^^T` lifts the *name* `T` into a `constexpr` value (`std::meta::info`)
- `[: info :]` drops a `std::meta::info` value back into a *type, expression, or member*

Why not earlier?

Why not earlier? — C++ libraries pre-P2996

Library	Type	Intrusive?	Coverage
RTTI (typeid)	language	none	type identity only
Boost.PFR	header	none	aggregates only
magic_enum	header	none	enums (via __PRETTY_FUNCTION__)
RTTR	header	macro	fields, methods, inheritance
EnTT::meta	header	registration	data + funcs (games)
Boost.Describe	header	macro	members + bases
Qt MOC	codegen	Q_OBJECT	full + signals/slots
Unreal UHT	codegen	UCLASS	full + GC + Blueprint
SWIG	codegen	none	multi-language bindings

Why not earlier? — every other language has this

Language	Runtime	Compile-time	Idiom
Python	● deep	●	<code>inspect</code> , <code>getattr</code> , <code>decorators</code>
Java	● native	● processors	<code>java.lang.reflect</code>
JavaScript	● Proxy	●	<code>Reflect</code> , <code>Object.*</code>
Rust	● minimal	● proc-macros	<code>derive</code> , <code>serde</code> , <code>bevy_reflect</code>
C++26	● build manually	● P2996	<code>^^T</code> , <code>[: r :]</code> , <code>consteval</code>

C++ was the only major language without first-class reflection — until now.

Supported compilers

Compiler	Reflexion C++26	<code>template for</code> (expansion statement)
GCC 16.1+	✓ Almost done	✓
Clang (Bloomberg)	✓ Almost done	✓ (<code>-freflection-latest</code>)
Clang mainline	● Partial	● In progress
MSVC	✗	✗
EDG	● Partial	●

A running example

```
struct Point {  
    int    x;  
    int    y;  
    double z;  
    double norm() const { return ...; }  
    Point scale(double) const { return ...; }  
};
```

Enumerate `Point`'s fields

```
constexpr auto ctx = std::meta::access_context::current();  
constexpr auto fields = std::define_static_array(std::meta::nonstatic_data_members_of(^Point, ctx));  
template for (constexpr auto f : fields) { // expansion statement  
    std::println(  
        "    {} : {}",  
        std::meta::identifier_of(f),  
        std::meta::display_string_of(std::meta::type_of(f))  
    );  
}
```

Output:

```
x : int  
y : int  
z : double
```

No macros, no inheritance, no registration, non intrusive. Just the type.

Enumerate `Point`'s methods

```
constexpr auto members = std::define_static_array(std::meta::members_of(^Point, ctx));

template for (constexpr auto m : members) { // expansion statement
    if constexpr (is_exportable_member_function(m)) {
        std::print("{} {}(", std::meta::display_string_of(std::meta::return_type_of(m)),
                    std::meta::identifier_of(m));

        bool first = true;
        template for (constexpr auto param: std::define_static_array(std::meta::parameters_of(m))) {
            if (!first) {std::print(", ");}
            first = false;
            std::print("{} ", std::meta::display_string_of(std::meta::type_of(param)));
        }
        std::println(")");
    }
}
```

Output:

```
double norm()
Point scale(double)
```

Simple JSON-ish serialization

```
template <typename T>
std::string to_json(const T& obj) {
    std::string out = "{";
    bool first = true;
    template for (constexpr auto m : std::define_static_array(
        std::meta::nonstatic_data_members_of(^T))) {
        if (!first) out += ",";
        first = false;
        out += '"';
        out += std::meta::identifier_of(m);
        out += "\":";
        out += serialize_value(obj.[:m:]); // user per-type value formatter
    }
    out += "}";
    return out;
}
```

Serialization example

```
enum class Color { Red, Green, Blue };

struct Address {
    std::string street;
    int         number;
};

struct Person {
    std::string      name;
    int              age;
    bool             active;
    Color            favorite_color;
    Address          home;
    std::vector<std::string> tags;
};

int main() {
    Person p{
        .name      = "Alice",
        .age       = 30,
        .active    = true,
        .favorite_color = Color::Green,
        .home      = {"Rue Pasteur", 42},
        .tags      = {"admin", "ops"},
    };

    std::println("{} ", to_json(p));
}
```

Serialization result

```
{
  "name": "Alice",
  "age": 30,
  "active": true,
  "favorite_color": "Green",
  "home": {
    "street": "Rue Pasteur",
    "number": 42
  },
  "tags": [
    "admin",
    "ops"
  ]
}
```

Aggregate example

Use `std::meta::define_aggregate` :

(previsouly called `std::meta::define_class` in draft)

```
#include <meta>

struct Synthesized; // incomplete – to be defined

consteval {
    std::meta::define_aggregate(^Synthesized, {
        std::meta::data_member_spec(^int, { .name = "id" }),
        std::meta::data_member_spec(^double, { .name = "value" }),
    });
}

// Synthesized is now:
struct Synthesized {int id; double value;};
```

Etonish, nein ? 😊 (cf, "*La Minute nécessaire de monsieur Cyclopède*")

Better Etonish!

```
constexpr auto v = [: parse_json(json_data) :];
```

Means: "Run `parse_json(json_data)` at **compile time**, obtain a reflected representation of some C++ entity or expression, and substitute the corresponding C++ code here."

The example of Rosetta

 rosetta2

- A C++ introspection & automatic language binding.
- Non intrusif
- Introspection **at runtime**
- Hand written registration à la `pybind11`

<https://github.com/xaliphostes/rosetta2>

Rosetta — *before* C++26

For each class, hand-write the registration:

```
rosetta::Class<Shape>("Shape")
    .field("name", &Shape::name).doc("display name")
    .method("describe", &Shape::describe).doc("greeting")
    .method("area", &Shape::area)
    .static_method("next_id", &Shape::next_id);

rosetta::Class<Circle>("Circle")
    .base<Shape>()
    .field("radius", &Circle::radius)
        .doc("radius in meters")
        .range(0.0, 1e6);

// ... repeat for Rectangle, every other type, every project ...
```

Real numbers from our internal lib: ~**6000** lines of glue.

Rosetta — *with* C++26

Use C++ annotations (P3394) for doc, readonly, numeric range, alias, displayed-name...

```
rosetta::register_reflected<Shape>();  
rosetta::register_reflected<Circle>();  
rosetta::register_reflected<Rectangle>();
```

That's it:

- Same registry
- Simplified downstream generators

~100 lines of `register_reflected` machinery — once, in the library — and *you never touch it again!*

Rosetta before and after C++26 (# lines of code)

Spec	Before	After
core	5,903	106
common	2,380	0
js	1,808	204
py	1,553	90
rest	3,327	305
wasm	1,435	102
Total	16,500	807

Applications — Auto language bindings

One C++ description, **N hosts** (visitors):

Target	What reflection drives
Python	pybind11 <code>def(...)</code> , <code>__init__</code> , dunder methods
JavaScript	Node.js N-API wrappers, getters/setters
WebAssembly	Emscripten <code>EMSCRIPTEN_BINDINGS</code>
TypeScript	<code>.d.ts</code> declaration files
REST API	HTTP routes mapped to methods, JSON I/O
Lua / Ruby / Julia	Sol2, Rice, CxxWrap
Java/JNI, C#/.NET, Swift	bridge generation

The alternative is **N hand-maintained binding files** that drift as the C++ API evolves.

Applications — Validation · Docs · Persistence · Live

- **Validation** — range, regex, not-null, cross-field invariants (from annotations).
- **Documentation** — Markdown / HTML / OpenAPI / Sphinx from `doc ("...")` .
- **Persistence / ORM (object relational mapping)** — `CREATE TABLE` , migrations from class diffs.
- **Configuration & DI (dependency injection)** — config-file → object hydration, CLI flag generation.
- **Live scripting / REPL (read-eval-print loop)** — every method auto-callable from the console.
- **Testing** — property-based, fuzzing, snapshot tests, binding-coverage.

Applications — Serialization · GUI · REST

Serialize: JSON / XML / YAML / TOML / Protobuf / ... — *no schema needed.*

GUI generation:

- Qt property editors (`ObjectInspector<Circle>` for free)
- QML data binding, web forms, Blueprint-style node editors
- ImGui debug panels — sliders, color pickers, drag-floats per field
- ...

REST / RPC: each method → endpoint, each parameter → JSON field.

```
for (auto cls : registry.list_classes())  
    for (auto m : cls->methods())  
        router.add(cls->name + "/" + m.name, &dispatch);
```


Annotation is the leverage

One `[[=doc{...}, =range{...}]] double radius;` is **simultaneously**:

- A Python attribute
- A JavaScript getter/setter
- A REST endpoint
- A JSON-serializable element
- A Qt property in the inspector
- A documented entry in the reference
- A range-validated database column
- ...

Every line of registration unlocks features across multiple subsystems at once.

QML example

```
using namespace rosetta;

struct Person {
    [[ = doc{"the person's display name"} ]]
    std::string name;

    [[ = doc{"age in whole years"},
      = range{0.0, 150.0},
      = widget::slider ]]
    int age = 0;

    [[ = doc{"server-assigned identifier"},
      = readonly{} ]]
    std::string id;

    [[ = rosetta::doc{"favourite colour"},
      = rosetta::combobox>{"red", "green", "blue", "yellow"} ]]
    std::string color = "red";

    Person() = default;
    Person(std::string n, int a, std::string i): name(std::move(n)), age(a), id(std::move(i)) {}

    [[ = doc{"Returns a greeting prefixed by the given salutation."} ]]
    std::string greet(const std::string &salutation) const {
        return salutation + ", " + name + "!";
    }

    void clear() {
        name.clear();
        age = 0;
    }
};

Person person;
rosetta::ReflectedObject reflected;
rosetta::bind_qml<Person>(&reflected, person);

QqmlApplicationEngine engine;
engine.rootContext()->setContextProperty("inspector", &reflected);
```



Qt example

```
class Algo {
public:

    [[ = rosetta::doc{"Tolerance of the solver"},
      = rosetta::range{1e-10, 1e-6},
      = rosetta::widget::textfield, = rosetta::label{"EPS"} ]]
    double eps{1e-7};

    [[ = rosetta::doc{"Set the max iterations"},
      = rosetta::range{0, 200},
      = rosetta::widget::slider, = rosetta::label{"Max iter"} ]]
    int maxIter{100};

    [[ = rosetta::doc{"Tell if the solver must be iterative"},
      = rosetta::widget::checkbox,
      = rosetta::label{"Iterative solver"} ]]
    bool iterative{true};

    [[ = rosetta::doc{"Solver name"},
      = rosetta::combobox{{"Seidel", "Jacobi", "gmres", "cgnr", "direct"}},
      = rosetta::label{"Solver name"} ]]
    std::string solverName{"Seidel"};

    [[ = rosetta::doc{"Run the solver and return the convergence"},
      = rosetta::button{"Run"} ]]
    double run() {return 1e-8;}

    [[ = rosetta::doc{"Reset the solver"},
      = rosetta::button{"Reset"} ]]
    void reset() {}
};
```

```
Algo algo;
QWidget *inspector = rosetta::build_inspector<Algo>(algo, "Algo");
```

Qt example



Without annotation



With annotation

Doc generation example

```
#include <rosetta/docgen.h>
#include "Algo.h"

const auto md = rosetta::generate_markdown<Algo>();
```

Algo (markdown preview)

Fields

Name	Type	Description
<code>eps</code>	<code>double</code>	Tolerance of the solver (range: 1e-10..1e-06)
<code>maxIter</code>	<code>int</code>	Set the max iterations (range: 0..200)
<code>iterative</code>	<code>bool</code>	Tell if the solver must be iterative
<code>solverName</code>	<code>std::string</code>	Solver name (choices: Seidel, Jacobi, gmres, cgnr)

Methods

`run()` → `double`

Run the solver and return the convergence

`reset()` → `void`

Playing with C++26 to recreate the Qt moc

Qt's moc is a separate code generator. It parses headers, emits .moc files, links them in. It's been a Qt fixture for 25 years.

C++26 reflection makes it... optional.

```
using namespace rosetta;

class Thermostat {
public:
    [[ = moc::signal ]]
    moc::Signal<double> temperatureChanged;

    [[ = moc::property{"temperature", "temperatureChanged"} ]]
    double m_temperature = 20.0;
};
```

No .moc file. No build step. Just a header.

```
Thermostat th;  
Display     d;  
  
moc::connect<"temperatureChanged", "showTemperature">(th, d);  
  
moc::set<"temperature">(th, 19.8); // fires temperatureChanged(19.8)  
moc::set<"temperature">(th, 19.8); // equality-gated, silent
```

Three failure modes, all at compile time:

- typo'd signal name → static_assert: "no [[=signal]]-tagged member"
- typo'd slot name → static_assert: "no [[=slot]]-tagged member"
- mismatched types → template error at the lambda body

References

- [Reflection proposal](#)
- [Annotations for reflection](#)
- [Python Bindings with Value-Based Reflection](#)
- [Using Reflection to Replace a Metalanguage for Generating JS Bindings](#)

Questions?

Example Python binding

```
template <typename T> void bind(py::module_ &m, const char *py_name) {
    py::class_<T> cls(m, py_name);
    cls.def(py::init<>());

    // Fields
    template for (constexpr auto fld : std::define_static_array(std::meta::nonstatic_data_members_of(^T, ctx))) {
        if constexpr (ro) {
            // readonly annotation -> Python read-only property
            cls.def_property_readonly(name, [] (const T &self) -> MemberT {
                return self.[:fld:]; }, docstr);
        }
    }

    // Methods
    template for (constexpr auto fn : std::define_static_array(std::meta::members_of(^T, ctx))) {
        if constexpr (is_exportable_member_function(fn)) {
            constexpr auto name = std::define_static_string(std::meta::identifier_of(fn));
            constexpr auto m_doc = std::meta::annotation_of_type<doc>(fn);
            constexpr const char *mdoc = m_doc.has_value() ? m_doc->text : "";

            if constexpr (std::meta::is_static_member(fn)) {
                cls.def_static(name, &[:fn:], mdoc);
            } else {
                cls.def(name, &[:fn:], mdoc);
            }
        }
    }
}
```

Calling a method

```
struct Vector3 {
    double x, y, z;
    double length() const { return x*x + y*y + z*z; }
    void scale(double k) { x *= k; y *= k; z *= k; }
};

constexpr auto ctx = std::meta::access_context::current();

// call a 0-arg method chosen by runtime name
double call_by_name(const Vector3 &v, std::string_view name) {
    double result = 0;
    // unrolls the member list at compile time
    template for (constexpr auto m : std::define_static_array(std::meta::members_of(^Vector3, ctx)))
    {
        if constexpr (std::meta::is_function(m) && !std::meta::is_special_member_function(m))
        {
            if (std::meta::identifier_of(m) == name && std::meta::parameters_of(m).empty())
            {
                result = (v.[:m:])(); // :-)
            }
        }
    }
    return result;
}

int main() {
    Vector3 v{3.0, 4.0, 0.0};
    std::println("{} ", call_by_name(v, "length")); // -> 25
}
```